Assignment 1

Name: Riddhi Agarwal
Roll Number: 2024CSB046

Problem 1

Question

Write a C program that dynamically creates an array of integers of size n, where n is a positive integer such that $n \geq 10$. The value of n must be read from the user at runtime. Populate the array with randomly generated integer values. After generating the array, design and implement an efficient algorithm to determine the 4th largest and the 3rd smallest elements present in the array. The solution should avoid sorting the entire array and must operate in linear time complexity, $O(n)$. Finally, display the generated array along with the computed 4th largest and 3rd smallest elements as output.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>

int thirdSmallest(int *arr, int n)
{
  int s1 = INT_MAX;
  int s2 = INT_MAX;
  int s3 = INT_MAX;

  for (int i = 0; i < n; i++)
  {
    if (arr[i] <= s1)
    {
      s3 = s2;
      s2 = s1;
      s1 = arr[i];
    }
    else if (arr[i] <= s2)
    {
      s3 = s2;
      s2 = arr[i];
    }
    else if (arr[i] < s3)
    {
      s3 = arr[i];
    }
```

```
  }
  return s3;
}

int fourthLargest(int *arr, int n)
{
  int l1 = INT_MIN;
  int l2 = INT_MIN;
  int l3 = INT_MIN;
  int l4 = INT_MIN;

  for (int i = 0; i < n; i++)
  {
    if (arr[i] >= l1)
    {
      l4 = l3;
      l3 = l2;
      l2 = l1;
      l1 = arr[i];
    }
    else if (arr[i] >= l2)
    {
      l4 = l3;
      l3 = l2;
      l2 = arr[i];
    }
    else if (arr[i] >= l3)
    {
      l4 = l3;
      l3 = arr[i];
    }
    else if (arr[i] > l4)
    {
      l4 = arr[i];
    }
  }
  return l4;
}

int main()
{
  srand(time(NULL));
  int n;
  do
  {
    printf("Enter the size(>=10) of the array: ");
    scanf("%d", &n);
    if (n < 10)
      printf("Size must be at least 10. Try again.\n");
  } while (n < 10);
  int *arr = (int *)malloc(sizeof(int) * n);
```

```c
  if (arr == NULL)
  {
    printf("Memory allocation failed.\n");
    return 1;
  }
  for (int i = 0; i < n; i++)
  {
    arr[i] = rand();
  }
  printf("The generated array with %d elements is:\n", n);
  for (int i = 0; i < n; i++)
  {
    printf("%d\t", arr[i]);
  }

  printf("\nThe 4th largest number in the array is %d.\n",
   fourthLargest(arr, n));
  printf("The 3rd smallest number in the array is %d.\n",
   thirdSmallest(arr, n));

  free(arr);

  return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> gcc Ques1.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> ./a.exe
Enter the size(>=10) of the array: 5
Size must be at least 10. Try again.
Enter the size(>=10) of the array: 10
The generated array with 10 elements is:
32452   7338    9925    9402    16124   14599   8417    5649    25658   5309
The 4th largest number in the array is 14599.
The 3rd smallest number in the array is 7338.
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2>
```

Figure 1: Output for Problem 1.

Analysis

The algorithm finds the 4th largest and 3rd smallest elements in a single pass ($O(n)$ time complexity) without sorting the array. It maintains a fixed-size set of variables (4 for largest, 3 for smallest) to track the extreme values encountered so far. Since the number of comparisons per element is constant, the space complexity is $O(1)$ (auxiliary space) and time complexity is $O(n)$.

Problem 2

Question

In this assignment, you are required to develop two separate programs to determine the maximum and minimum elements in an array of n integers. The first program should implement the naïve Max-Min algorithm, which finds the maximum and minimum values by scanning the array sequentially. The second program should implement the Max-Min algorithm using the Divide and Conquer strategy, where the array is recursively divided into smaller subarrays and the maximum and minimum elements are determined efficiently. Execute both programs on randomly generated input arrays of varying sizes, specifically: $n = 100, 1000, 10,000, 20,000, \ldots, 100,000$. For each input size, count and record the number of element comparisons performed by each algorithm. Finally, plot a graph with the input size (n) on the X-axis and the number of comparisons on the Y-axis to visually compare the performance of the two approaches.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int naive_comparisons = 0;
int dc_comparisons = 0;

struct MinMax
{
    int min;
    int max;
};

struct MinMax getMinMaxNaive(int arr[], int n)
{
    struct MinMax result;
    naive_comparisons = 0;

    if (n == 1)
    {
        result.max = arr[0];
        result.min = arr[0];
        return result;
    }

    if (arr[0] > arr[1])
    {
        result.max = arr[0];
        result.min = arr[1];
    }
    else
    {
        result.max = arr[1];
```

```
        result.min = arr[0];
    }
    naive_comparisons++;

    for (int i = 2; i < n; i++)
    {
        naive_comparisons++;
        if (arr[i] > result.max)
        {
            result.max = arr[i];
        }
        else
        {
            naive_comparisons++;
            if (arr[i] < result.min)
            {
                result.min = arr[i];
            }
        }
    }
    return result;
}

struct MinMax getMinMaxDC(int arr[], int low, int high)
{
    struct MinMax result, left, right;

    if (low == high)
    {
        result.max = arr[low];
        result.min = arr[low];
        return result;
    }

    if (high == low + 1)
    {
        dc_comparisons++;
        if (arr[low] > arr[high])
        {
            result.max = arr[low];
            result.min = arr[high];
        }
        else
        {
            result.max = arr[high];
            result.min = arr[low];
        }
        return result;
    }

    int mid = (low + high) / 2;
```

```c
    left = getMinMaxDC(arr, low, mid);
    right = getMinMaxDC(arr, mid + 1, high);

    dc_comparisons++;
    if (left.min < right.min)
        result.min = left.min;
    else
        result.min = right.min;

    dc_comparisons++;
    if (left.max > right.max)
        result.max = left.max;
    else
        result.max = right.max;

    return result;
}

int main()
{
    srand(time(0));
    int sizes[] = {100, 1000, 10000, 20000, 30000, 40000, 50000, 60000, 70000, 80000, 90000, 100000};
    int num_tests = sizeof(sizes) / sizeof(sizes[0]);

    FILE *fp = fopen("data.txt", "w");
    if (fp == NULL) {
        printf("Error: Could not create data.txt\n");
        return 1;
    }

    printf("%-10s | %-20s | %-20s\n", "Input Size", "Naive Comparisons", "D&C Comparisons");
    printf("------------------------------------------------------------\n");

    for (int t = 0; t < num_tests; t++)
    {
        int n = sizes[t];
        int *arr = (int *)malloc(n * sizeof(int));

        for (int i = 0; i < n; i++)
            arr[i] = rand();

        getMinMaxNaive(arr, n);

        dc_comparisons = 0;
        getMinMaxDC(arr, 0, n - 1);

        printf("%-10d | %-20d | %-20d\n", n, naive_comparisons, dc_comparisons);
```

```c
        fprintf(fp, "%d %d %d\n", n, naive_comparisons, dc_comparisons);

        free(arr);
    }

    fclose(fp);
    printf("\nSuccess! Data saved to 'data.txt'.\n");

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> gcc Ques2.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> ./a.exe
Input Size | Naive Comparisons    | D&C Comparisons
-----------------------------------------------------------
100        | 196                  | 162
1000       | 1989                 | 1510
10000      | 19987                | 15902
20000      | 39988                | 31806
30000      | 59988                | 46382
40000      | 79986                | 63614
50000      | 99991                | 82766
60000      | 119988               | 92766
70000      | 139993               | 107230
80000      | 159993               | 127230
90000      | 179985               | 147230
100000     | 199990               | 165534

Success! Data saved to 'data.txt'.
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> gnuplot

        G N U P L O T
        Version 6.0 patchlevel 4    last modified 2025-12-18

        Copyright (C) 1986-1993, 1998, 2004, 2007-2025
        Thomas Williams, Colin Kelley and many others

        gnuplot home:     http://www.gnuplot.info
        faq, bugs, etc:   type "help FAQ"
        immediate help:   type "help"  (plot window: hit 'h')

        Terminal type is now qt
Encoding set to 'cp1252'.
gnuplot> plot "data.txt" using 1:2 w lp title "Naive", "data.txt" using 1:3 w lp title "Divide and Conquer"
gnuplot> exit
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2>
```

Figure 2: Output for Problem 2.

Analysis

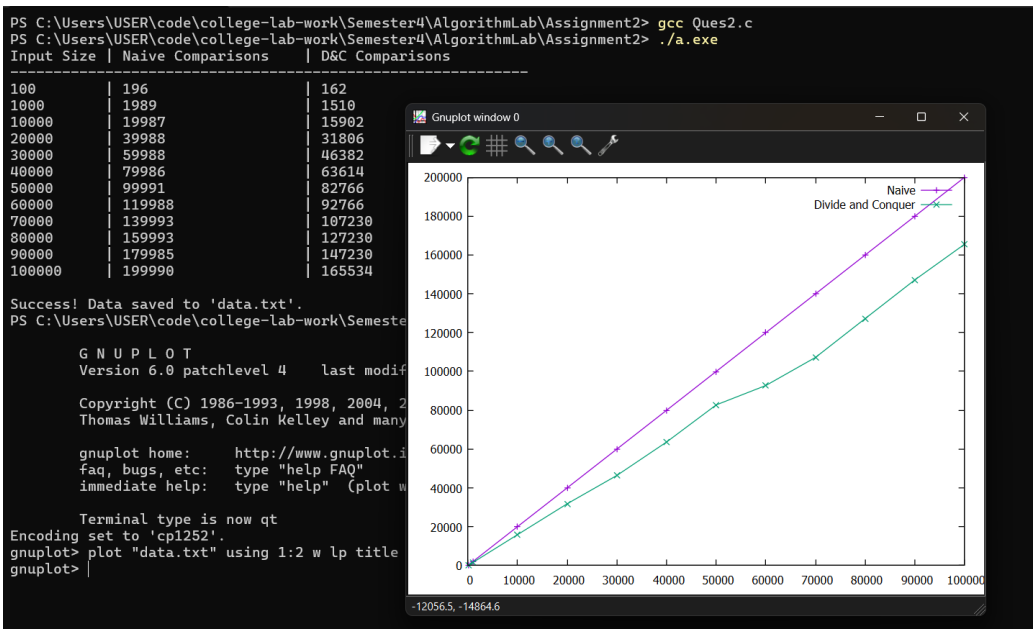

Figure 3: Comparison of Naive vs. Divide and Conquer performance.

The Naive algorithm performs roughly $2(n-1)$ comparisons in the worst case. In contrast, the Divide and Conquer approach reduces this to $\frac{3n}{2} - 2$ comparisons. As seen in the graph, the green line (D&C) grows at a lower slope than the purple line (Naive), demonstrating that for large datasets, the Divide and Conquer strategy is significantly more efficient in terms of the number of element comparisons.

Problem 3

Question

Consider a source string $S[0 \ldots n-1]$ of length n, composed exclusively of the symbols 'c' and 'd'. You are also given a pattern string $P[0 \ldots m-1]$ of length m, where $m \leq n$. The pattern string consists of the symbols 'c', 'd', and the special character '*'. The symbol '*' acts as a wildcard and can match any single character, either 'c' or 'd', in the source string. All other symbols in the pattern must match the corresponding characters in the source string exactly. The objective is to identify all valid match positions in the source string where the pattern occurs. A position j is considered a valid match if the pattern P matches the substring $S[j \ldots j+m-1]$. The output should be a sorted list M containing all such valid starting indices j in the source string.

Example: If the source string is $S = ccdcddcd$ and the pattern is $P = cd*$, then the pattern matches the source string starting at positions 0 and 2. Therefore, the output list M should be $[0, 2]$.

Code

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void matchPattern(char *source, char *pattern)
{
    int n = strlen(source);
    int m = strlen(pattern);
    int found = 0;

    printf("Valid match positions: [ ");
    for (int i = 0; i <= n - m; i++)
    {
        int j;
        for (j = 0; j < m; j++)
        {
            if (pattern[j] != '*' && pattern[j] != source[i + j])
            {
                break;
            }
        }
        if (j == m)
        {
            printf("%d ", i);
            found = 1;
        }
    }
    printf("]\n");

    if (!found)
    {
```

```c
        printf("No matches found.\n");
    }
}

int main()
{
    char S[1000], P[1000];

    printf("Enter Source String S (only 'c' and 'd'): ");
    scanf("%s", S);

    printf("Enter Pattern String P ('c', 'd', '*'): ");
    scanf("%s", P);

    matchPattern(S, P);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> gcc Ques3.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> ./a.exe
Enter Source String S (only 'c' and 'd'): cdcddcd
Enter Pattern String P ('c', 'd', '*'): cd*
Valid match positions: [ 0 2 ]
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2>
```

Figure 4: Output for Problem 3.

Problem 4

Question

Design an algorithm to create two sets of integers, namely A and B. The number of elements in each set must be entered by the user at runtime. While constructing the sets, ensure that each set contains only distinct elements, thereby preventing the insertion of duplicate values. After successfully creating the two sets, develop a procedure to determine whether set A is a subset of set B. Additionally, verify whether A is a proper subset of B.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct
{
    int *elements;
    int size;
    int capacity;
} Set;

void initSet(Set *s, int cap)
{
    s->elements = (int *)malloc(cap * sizeof(int));
    s->size = 0;
    s->capacity = cap;
}

bool exists(Set *s, int val)
{
    for (int i = 0; i < s->size; i++)
    {
        if (s->elements[i] == val)
            return true;
    }
    return false;
}

void add(Set *s, int val)
{
    if (!exists(s, val))
    {
        if (s->size < s->capacity)
        {
            s->elements[s->size++] = val;
        }
    }
}
```

```c

void createSetFromInput(Set *s, char name)
{
    int n, val;
    printf("Enter number of elements for Set %c: ", name);
    scanf("%d", &n);

    initSet(s, n);

    printf("Enter %d integers for Set %c: ", n, name);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &val);
        add(s, val);
    }
}

bool isSubset(Set *A, Set *B)
{
    for (int i = 0; i < A->size; i++)
    {
        if (!exists(B, A->elements[i]))
        {
            return false;
        }
    }
    return true;
}

int main()
{
    Set A, B;

    createSetFromInput(&A, 'A');
    createSetFromInput(&B, 'B');

    bool sub = isSubset(&A, &B);

    if (sub)
    {
        printf("\nSet A is a subset of Set B.\n");
        if (A.size < B.size)
        {
            printf("Set A is a proper subset of Set B.\n");
        }
        else
        {
            printf("Set A is not a proper subset of Set B (The sets are equal).\n");
        }
    }
```

```c
    else
    {
        printf("\nSet A is not a subset of Set B.\n");
        printf("Set A is not a proper subset of Set B.\n");
    }

    free(A.elements);
    free(B.elements);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> gcc Ques4.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> ./a.exe
Enter number of elements for Set A: 3
Enter 3 integers for Set A: 1 2 3
Enter number of elements for Set B: 4
Enter 4 integers for Set B: 1 2 3 4

Set A is a subset of Set B.
Set A is a proper subset of Set B.
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> ./a.exe
Enter number of elements for Set A: 3
Enter 3 integers for Set A: 1 2 3
Enter number of elements for Set B: 3
Enter 3 integers for Set B: 1 2 3

Set A is a subset of Set B.
Set A is not a proper subset of Set B (The sets are equal).
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2>
```

Figure 5: Output for Problem 4.

Problem 5

Question

You are given k disjoint or overlapping sets $S_1, S_2, \ldots, S_k$ each containing up to n integers. Design an algorithm to compute the union of all k sets.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct
{
    int *elements;
    int size;
    int capacity;
} Set;

void initSet(Set *s, int cap)
{
    s->elements = (int *)malloc(cap * sizeof(int));
    s->size = 0;
    s->capacity = cap;
}

bool exists(Set *s, int val)
{
    for (int i = 0; i < s->size; i++)
    {
        if (s->elements[i] == val)
            return true;
    }
    return false;
}

void add(Set *s, int val)
{
    if (!exists(s, val))
    {
        if (s->size < s->capacity)
        {
            s->elements[s->size++] = val;
        }
    }
}

void createSetFromInput(Set *s, int index)
{
```

```c
    int n, val;
    printf("Enter number of elements for Set %d: ", index);
    scanf("%d", &n);

    initSet(s, n);

    printf("Enter %d integers for Set %d: ", n, index);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &val);
        add(s, val);
    }
}

int main()
{
    int k;
    printf("Enter the number of sets (k): ");
    scanf("%d", &k);

    if (k < 1)
    {
        printf("k must be at least 1.\n");
        return 1;
    }

    Set *sets = (Set *)malloc(k * sizeof(Set));
    int total_capacity_needed = 0;

    for (int i = 0; i < k; i++)
    {
        createSetFromInput(&sets[i], i + 1);
        total_capacity_needed += sets[i].size;
    }

    Set unionSet;
    initSet(&unionSet, total_capacity_needed);

    for (int i = 0; i < k; i++)
    {
        for (int j = 0; j < sets[i].size; j++)
        {
            add(&unionSet, sets[i].elements[j]);
        }
    }

    printf("\nUnion of all %d sets: { ", k);
    for (int i = 0; i < unionSet.size; i++)
    {
        printf("%d ", unionSet.elements[i]);
    }
```

```c
    printf("}\n");

    for (int i = 0; i < k; i++)
        free(sets[i].elements);
    free(sets);
    free(unionSet.elements);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> gcc Ques5.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2> ./a.exe
Enter the number of sets (k): 3
Enter number of elements for Set 1: 1
Enter 1 integers for Set 1: 1
Enter number of elements for Set 2: 2
Enter 2 integers for Set 2: 1 2
Enter number of elements for Set 3: 5
Enter 5 integers for Set 3: 1 2 3 4 5

Union of all 3 sets: { 1 2 3 4 5 }
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment2>
```

Figure 6: Output for Problem 5.